Technology & Feasibility

AI-Based Restoration of Degraded Images for Semiconductor Inspection

1. Core ML Stack

Deep Learning Framework

Framework: PyTorch 2.x with torch.cuda.amp (mixed-precision training)

Tensor Ops: torchvision (image transforms), torch.fft (FFT loss)

Restoration Model

Architecture: NAFNet-Tiny (Nonlinear Activation Free Network)

Paper: Chen et al., "Simple Baselines for Image Restoration", ECCV 2022

Parameters: 0.48M (480K trainable parameters)

Input: 1-channel grayscale (adapted from 3-ch RGB)

Structure: UNet encoder-decoder with skip connections

Key Blocks: LayerNorm2d -> DWConv3x3 -> SimpleGate -> SCA -> FFN

SR Head: PixelShuffle(2) for 2x super-resolution

Residual: Global residual: output = f(x) + bicubic_upsample(x)

Loss Functions (Non-GAN, Pixel-Fidelity Only)

Charbonnier: sqrt((pred - target)^2 + eps^2), eps=1e-3 (robust smooth L1)

SSIM Loss: 1 - SSIM(pred, target), window_size=11, gaussian kernel

Composite: L = 1.0 * L_Charbonnier + 0.1 * L_SSIM

2. Data Pipeline

Dataset: KLA Semiconductor Paired Dataset (.npy format)

Pairing: Degraded (noisy LR) <-> Ground Truth (clean HR), matched filenames

Loading: NumPy .npy loader with auto-detection for .png/.tif fallback

Normalization: Input: x / max(x) (handles speckle overflow); Target: x / 255.0

Patch Cropping: Random crop: 64x64 LR -> 128x128 HR (at 2x scale)

Augmentation: Random horizontal flip + vertical flip + 90-degree rotation

Validation: Last 10% of training pairs held out for PSNR/SSIM tracking

3. Training Configuration

Parameter	Value	Rationale
Optimizer	AdamW	Stable, weight-decoupled
Learning Rate	1e-3	NAFNet default, proven effective
Weight Decay	1e-4	Mild regularization
Scheduler	Cosine Annealing	Smooth LR decay to zero

Warmup	500 iterations	Linear warmup from 1e-6 to prevent instability
Batch Size	8	Fits T4 16GB VRAM comfortably
Patch Size	64x64 (LR)	HR target = 128x128 at 2x scale
Max Iterations	15,000	~2.5 hours on T4 GPU
Precision	FP16 (AMP)	torch.cuda.amp for 1.5-2x speedup
Gradient Clip	max_norm=1.0	Prevents gradient explosion
Checkpointing	Every 1,000 iters	Saved to Google Drive (Colab survival)
Validation	Every 500 iters	Track PSNR/SSIM on held-out split

4. Infrastructure & Compute

Platform: Google Colab (free tier or Pro)

GPU: NVIDIA Tesla T4 (16 GB VRAM, Turing architecture)

Training Time: ~2.5 hours for 15K iterations on T4

Storage: Google Drive for checkpoints + dataset (persistent across sessions)

Checkpoint Resume: Full state save (model, optimizer, scheduler, iteration, best PSNR)

5. Model Specifications

Metric	Value	Notes
Parameters	0.48M (480K)	NAFNet-Tiny: width=16, enc=[1,1,1,2]
Model File Size	~1.9 MB (.pt)	Easily fits in GitHub (<25MB limit)
GPU Inference	<3 ms / 256x256	>300 FPS, real-time capable
CPU Inference	~20 ms / 256x256	~50 FPS, interactive
Input Format	1-ch grayscale float32	Normalized to [0, 1]
Output Format	1-ch grayscale uint8	[0, 255] or .npy float32
Scale Factor	2x	Joint denoise + super-resolution
Export	**ONNX-compatible**	**No custom ops, direct export**

6. Evaluation & Metrics

PSNR: 10 * log10(1 / MSE) in [0,1] range -- higher is better

SSIM: Structural Similarity Index -- higher is better (max 1.0)

LPIPS: Learned Perceptual Similarity -- lower is better (optional)

Eval Script: evaluate.py: standalone, auto-detects format, handles arbitrary sizes

Dummy Baseline: Bicubic upsampling fallback (works without trained weights)

7. Software Dependencies

Package	Version	Purpose
torch	>=2.0.0	Deep learning framework, model training & inference

torchvision	>=0.15.0	Image transforms and utilities
numpy	>=1.24.0	.npy data loading, array operations
Pillow	>=9.0.0	Image I/O for .png/.tif formats
scipy	>=1.10.0	Gaussian kernel generation for SSIM
pyyaml	>=6.0	Config file parsing (default.yaml)
tqdm	>=4.65.0	Training progress bars
opencv-python-headless	>=4.7.0	Image preprocessing, resizing
scikit-image	>=0.20.0	SSIM metric computation for evaluation

8. Repository Structure

```
semicon-restore/
  configs/default.yaml        # All hyperparameters
  data/dataset.py             # KLA paired .npy loader + augmentations
  models/
    nafnet.py                  # NAFNet backbone (LayerNorm, SimpleGate, SCA)
    nafnet_sr.py               # Joint Denoise + 2x SR with PixelShuffle
    losses.py                  # Charbonnier + SSIM composite loss
  weights/nafnet_tiny.pt       # Trained model weights (~1.9 MB)
  outputs/                     # Restored test images
  evaluate.py                  # Standalone eval script (scored by KLA)
  train.py                     # Training with checkpoint resume + AMP
  train_colab.ipynb            # Google Colab training notebook
  requirements.txt             # Python dependencies
  README.md                    # Setup & quickstart guide
```

GitHub: https://github.com/Luv120/Semigone